

IASI R User Guide

Build, publication and release for IASI projects

IASI Project

Table of contents

1	IASI R User Guide	4
2	Definiciones y conceptos	5
2.1	Definiciones y conceptos	5
3	Instalación	7
3.1	Requisitos	7
3.2	Instalar <code>iasi</code>	7
3.3	Comprobar la instalación	8
4	Tipos de publicación	9
4.1	Proyectos Quarto y estrategias	9
4.2	Estrategias soportadas	9
4.3	Configuración de un libro	10
4.4	Crear un Website	11
4.5	Crear un Book Regular	11
4.6	Crear un Book Structured	12
4.7	Crear un Book Parted	13
4.8	Crear un Book Direct	14
4.9	Elegir el modo de publicación	15
5	Formatos	16
5.1	Formatos disponibles	16
5.2	Configurar un formato	16
5.3	Construir formatos	17
6	Construir y publicar	18
6.1	Operaciones principales	18
6.1.1	Validar	18
6.1.2	Construir	18
6.1.3	Publicar	19
6.1.4	Liberar	20
6.2	Flujo recomendado	21
7	Multiproyectos	23
7.1	Trabajar con varios proyectos IASI	23

7.2	Configuración heredada	23
7.3	<code>path</code> define el ámbito	24
7.4	Aplicabilidad de las operaciones	24
7.5	Publicación local por proyecto	25
7.6	<code>release()</code> reúne los entregables	25
8	Problemas frecuentes	26
8.1	El proyecto no se reconoce	26
8.2	Hay dos archivos de configuración IASI	26
8.3	Un proyecto hijo no se descubre	26
8.4	Aparece “No IASI publication strategy defined”	27
8.5	Un formato no se genera	27
8.6	Quiero reconstruir	27
8.7	<code>publish()</code> dice que no hay formatos contruidos	28
8.8	Quiero publicar lo que ya está construido	28
8.9	Quiero preparar los entregables locales	28
8.10	Quiero desplegar a un servicio externo	28

1 IASI R User Guide

`iasi-r` es el producto del ecosistema IASI para el entorno R. Su paquete R, `iasi`, descubre, valida, construye, publica y reúne entregables de proyectos IASI.

El proyecto nació como `iasi-quarto`, centrado inicialmente en publicaciones Quarto. Al ampliar su responsabilidad para incluir otros proyectos del entorno R, especialmente paquetes R, evolucionó a **IASI R**.

IASI R no sustituye a Quarto. Quarto sigue definiendo el proyecto documental, sus perfiles y sus formatos. IASI añade una capa de ingeniería alrededor del ciclo de vida: descubrimiento de proyectos, configuración heredable, preparación estructural, construcción, publicación local y release.

Esta guía explica **cómo utilizar IASI R**. No describe su arquitectura interna ni pretende documentar cada función privada.

El recorrido recomendado es sencillo:

1. conocer los términos utilizados por IASI R;
2. instalar el paquete `iasi`;
3. declarar el tipo de proyecto IASI;
4. decidir si un proyecto Quarto utiliza una estrategia de publicación IASI o Quarto as-is;
5. configurar los formatos de salida;
6. ejecutar `validate()`, `build()`, `publish()` y `release()` según corresponda;
7. utilizar un repositorio multiproyecto cuando varios proyectos comparten una misma raíz.

Si ya conoce IASI R, puede ir directamente al capítulo que corresponda a la tarea que quiere realizar.

2 Definiciones y conceptos

2.1 Definiciones y conceptos

En este documento se utilizan los siguientes términos y conceptos. Las definiciones se presentan en orden alfabético para facilitar su consulta.

Término	Definición
Build	Operación que construye los artefactos de los proyectos seleccionados. En proyectos Quarto materializa salidas en <code>_outputs/</code> ; en paquetes R genera los artefactos fuente y binario. Se ejecuta siempre que se invoca.
Capítulo	Unidad principal de contenido de un libro. Puede estar organizada en secciones.
Direct	Estrategia de libro en la que cada documento <code>.qmd</code> del contenido se publica como capítulo y las carpetas se utilizan únicamente para organizar físicamente los archivos.
Estrategia	Regla declarada en <code>publication.strategy</code> que determina cómo prepara IASI la estructura editorial de un proyecto Quarto antes del renderizado y cómo aplica su postprocesado.
Formato	Salida materializada por un perfil, por ejemplo HTML, PDF, EPUB, DOCX u ODT.
GitBook	Salida preparada para GitBook mediante el perfil lógico git.
IASI R	Producto IASI para el entorno R. Su paquete, <code>iasi</code> , implementa <code>validate()</code> , <code>build()</code> , <code>publish()</code> y <code>release()</code> .
Libro	Proyecto Quarto cuyo <code>project.type</code> es <code>book</code> .
Multi-proyecto	Ámbito que contiene varios proyectos IASI y permite descubrirlos y operar sobre ellos desde una raíz común.
Parte	División de alto nivel de un libro utilizada para agrupar capítulos relacionados.
Parted	Estrategia de libro en la que cada carpeta superior representa una Parte y puede contener capítulos directos o subcarpetas de capítulo con su propio <code>index.qmd</code> y documentos asociados.
Perfil	Configuración lógica de salida declarada por Quarto. IASI utiliza los perfiles declarados para decidir qué formatos puede construir y publicar.
Proyecto IASI	Directorio marcado por <code>_iasi.yml</code> o <code>.iasi.yml</code> y con un type IASI válido.

Término	Definición
Proyecto Quarto	Proyecto reconocido por Quarto mediante su configuración habitual, incluida <code>__quarto.yml</code> .
Publicación	Materialización local preparada por <code>publish()</code> a partir de artefactos ya construidos en <code>__outputs/</code> .
Publish	Operación que toma formatos construidos en <code>__outputs/</code> , los copia a un árbol temporal, aplica el postprocesado correspondiente y sustituye <code>__publish/</code> . No vuelve a renderizar.
Quarto	Motor que procesa documentos y genera las salidas configuradas por los proyectos Quarto.
Quarto as-is	Modo válido de type: quarto sin <code>publication.strategy</code> . IASI no prepara una estructura editorial propia y delega el renderizado en Quarto.
Regular	Estrategia de libro en la que cada carpeta de contenido se convierte en una unidad o capítulo y sus documentos <code>.qmd</code> se combinan en un <code>index.qmd</code> generado.
Release	Operación que reúne entregables locales en <code>release/</code> . Para software recoge artefactos construidos; para publicaciones recoge la materialización preparada o, en Quarto as-is, los outputs construidos.
Repository	Tipo IASI usado como raíz o contenedor de otros proyectos. Puede aportar configuración heredable, pero no se construye ni publica directamente.
R package	Tipo IASI para un paquete R. <code>build()</code> genera el paquete fuente y binario; <code>release()</code> recoge esos artefactos.
Sección	División principal dentro de un capítulo.
Structured	Estrategia de libro en la que cada carpeta de contenido representa una Parte y sus documentos <code>.qmd</code> directos son capítulos de esa Parte.
Validate	Operación que comprueba los proyectos IASI seleccionados sin construir, publicar ni liberar artefactos.
Website	Tipo IASI para un proyecto Quarto website publicable sin estrategia editorial de libro.

3 Instalación

3.1 Requisitos

Para utilizar IASI R necesita:

- R 4.2 o posterior;
- Quarto para los proyectos documentales;
- las dependencias adicionales de los formatos que quiera generar, por ejemplo una instalación TeX para PDF;
- Git si va a instalar el paquete directamente desde GitHub.

Puede comprobar R y Quarto desde una terminal:

```
R --version  
quarto --version
```

3.2 Instalar `iasi`

La implementación actual vive en el subproyecto `software/iasi-r` del repositorio `iasi-r`.

Una forma sencilla de instalarla desde GitHub es utilizar `remotes`:

```
install.packages("remotes")  
remotes::install_github(  
  "iasi-org-dev/iasi-r",  
  subdir = "software/iasi-r"  
)
```

Después puede cargar el paquete:

```
library(iasi)
```

3.3 Comprobar la instalación

Compruebe la versión instalada:

```
packageVersion("iasi")
```

Y, desde un proyecto IASI, ejecute:

```
validate()
```

Si el proyecto es válido, IASI R lo comprobará sin construir todavía ningún artefacto.

4 Tipos de publicación

4.1 Proyectos Quarto y estrategias

IASI R distingue entre el **tipo de proyecto IASI** y la configuración propia de Quarto.

Un libro gestionado por IASI se declara así en `_iasi.yml`:

```
type: quarto  
  
publication:  
  strategy: regular
```

Mientras que `_quarto.yml` continúa declarando el tipo Quarto:

```
project:  
  type: book
```

La estrategia no es una propiedad raíz del proyecto. Su ubicación canónica es siempre:

```
publication.strategy
```

Un `type: quarto` sin `publication.strategy` también es válido. En ese caso IASI utiliza **Quarto as-is**: no prepara una estructura editorial IASI antes del renderizado.

4.2 Estrategias soportadas

Cuando un libro opta por la preparación editorial de IASI, dispone de cuatro estrategias:

Es- trate- gia	Organización editorial	Cuándo utilizarla
reg- ular	Cada carpeta de contenido forma una unidad y sus archivos .qmd se combinan.	Cuando varios documentos fuente forman una única unidad editorial.
struc- tured	Cada carpeta superior representa una Parte; su index.qmd presenta la Parte y los demás .qmd son Capítulos.	Cuando la estructura Parte → Capítulo se representa directamente en cada carpeta superior.
parted	Cada carpeta superior representa una Parte y puede contener capítulos directos o subcarpetas de capítulo con su propio index.qmd.	Cuando necesita Partes con capítulos que a su vez pueden agrupar varios documentos.
di- rect	Cada archivo .qmd se publica directamente como Capítulo; las carpetas solo organizan los fuentes.	Cuando cada documento ya es una unidad editorial completa.

4.3 Configuración de un libro

Para un proyecto Quarto de tipo book, IASI proporciona valores predeterminados para la organización del contenido:

```
front-matter = front-matter
content-dir  = chapters
back-matter  = back-matter
numbered     = true
```

La estrategia **no tiene valor predeterminado**. Debe declararse explícitamente si desea utilizar el pipeline editorial de IASI.

Una configuración habitual puede ser:

```
type: quarto

publication:
  strategy: parted
  content-dir: chapters
  front-matter: front-matter
  back-matter: back-matter
  numbered: true
```

Cuando `numbered: true`, los documentos y carpetas que participan en el orden editorial utilizan prefijos numéricos, por ejemplo:

```
chapters/  
  01-intro/  
  02-installation/  
  03-usage/
```

4.4 Crear un Website

IASI R representa un Website como un tipo IASI propio. En `_iasi.yml`:

```
type: website
```

Y Quarto continúa describiendo el proyecto en `_quarto.yml`:

```
project:  
  type: website
```

Un Website no necesita `publication.strategy`. Su estructura y navegación siguen siendo las definidas por Quarto.

IASI R puede construirlo, preparar su árbol local de publicación y reunirlo posteriormente en `release/`.

Para construir su perfil web:

```
build()
```

4.5 Crear un Book Regular

`regular` combina varios documentos fuente de una carpeta en una única unidad editorial.

La idea es:

```
carpeta de contenido  
  → capítulo o unidad de publicación  
  
documentos .qmd dentro de la carpeta  
  → contenido combinado de esa unidad
```

Ejemplo:

```
chapters/  
  01-intro/  
    front-matter.quarto  
    01-purpose.qmd  
    02-scope.qmd
```

IASI R genera el `index.qmd` de la carpeta combinando los documentos fuente.

`front-matter.quarto` puede proporcionar el front matter del `index.qmd` generado:

```
---  
title: "Introducción"  
title-block-style: none  
---
```

Por tanto, **no edite el `index.qmd` generado**. Edite los documentos fuente y, cuando corresponda, `front-matter.quarto`.

En `_iasi.yml`:

```
type: quarto  
  
publication:  
  strategy: regular
```

Use **regular** cuando una carpeta represente una unidad editorial formada por varios fragmentos o documentos fuente.

4.6 Crear un Book Structured

`structured` representa explícitamente Partes y Capítulos.

La regla es:

```
carpeta de contenido  
  → Parte  
  
index.qmd de la carpeta  
  → presentación de la Parte  
  
demás .qmd de la carpeta  
  → Capítulos
```

Ejemplo:

```
chapters/  
  20-methodologies/  
    index.qmd  
    01-history.qmd  
    02-current.qmd
```

En `_iasi.yml`:

```
type: quarto  
  
publication:  
  strategy: structured  
  content-dir: chapters  
  numbered: true
```

A diferencia de **regular**, el `index.qmd` de cada carpeta es **fuentes** y debe existir. IASI R no lo genera a partir de otros documentos.

Use **structured** cuando quiera que la jerarquía editorial Parte → Capítulo esté representada directamente por la estructura del proyecto.

4.7 Crear un Book Parted

parted extiende el modelo de Partes para permitir capítulos compuestos por varios documentos.

La regla principal es:

```
carpeta superior  
  → Parte  
  
index.qmd de la carpeta superior  
  → presentación de la Parte  
  
.qmd directos  
  → Capítulos directos  
  
subcarpeta de capítulo  
  index.qmd  
  otros .qmd  
  → Capítulo + documentos asociados
```

Ejemplo:

```
chapters/  
  20-methodologies/  
    index.qmd  
    intro.qmd  
    01-history.qmd  
    02-current/  
      index.qmd  
      01-model.qmd  
      02-practice.qmd
```

En `_iasi.yml`:

```
type: quarto  
  
publication:  
  strategy: parted  
  content-dir: chapters  
  numbered: true
```

Cada Parte necesita su `index.qmd`. Las subcarpetas que representan capítulos también necesitan su propio `index.qmd`.

Use `parted` cuando la jerarquía editorial necesite **Partes y capítulos con estructura interna**, sin renunciar a documentos que funcionen como capítulos directos de una Parte.

4.8 Crear un Book Direct

`direct` publica cada documento `.qmd` como capítulo.

Las carpetas sirven para ordenar físicamente los fuentes, pero no crean Partes ni capítulos contenedores.

Ejemplo:

```
chapters/  
  01-concepts/  
    01-model.qmd  
    02-context.qmd  
  02-usage/  
    01-build.qmd  
    02-publish.qmd
```

La publicación resultante contiene los cuatro documentos como capítulos consecutivos.

En `_iasi.yml`:

```
type: quarto

publication:
  strategy: direct
  content-dir: chapters
  numbered: true
```

Use `direct` cuando cada `.qmd` ya sea una unidad editorial completa y las carpetas solo deban ayudar a mantener organizado el repositorio.

4.9 Elegir el modo de publicación

Si necesita...	Configuración
Un proyecto Quarto sin preparación estructural de IASI	<code>type: quarto</code> , sin <code>publication.strategy</code>
Un Website Quarto	<code>type: website</code>
Combinar varios archivos <code>.qmd</code> de una carpeta en una única unidad	<code>publication.strategy: regular</code>
Representar Partes con capítulos directos	<code>publication.strategy: structured</code>
Representar Partes con capítulos que pueden agrupar varios documentos	<code>publication.strategy: parted</code>
Publicar cada archivo <code>.qmd</code> directamente como Capítulo	<code>publication.strategy: direct</code>

La elección de estrategia es editorial. No depende del formato final: un mismo libro puede utilizar la misma estrategia para HTML, PDF, EPUB y otros perfiles.

5 Formatos

5.1 Formatos disponibles

IASI R utiliza los perfiles declarados por Quarto y dispone de renderers para los perfiles de libro habituales:

Perfil	Resultado habitual
html	Publicación HTML
pdf	PDF generado por Quarto
pdfua	PDF/UA mediante Typst
epub	Libro EPUB
docx	Documento DOCX
odt	Documento ODT
git	GitBook

Un proyecto IASI **website** utiliza su perfil lógico **web**. Un proyecto **quarto** de tipo Book puede declarar varios perfiles de salida.

El perfil **git** representa una publicación destinada a GitBook.

5.2 Configurar un formato

Cada perfil se configura mediante su configuración Quarto correspondiente, por ejemplo:

```
_quarto-html.yml  
_quarto-pdf.yml  
_quarto-epub.yml
```

IASI R no sustituye la configuración propia de Quarto para cada formato.

5.3 Construir formatos

Para construir solo HTML:

```
build(format = "html")
```

Para construir solo PDF:

```
build(format = "pdf")
```

En proyectos con estrategia IASI puede solicitar varios perfiles:

```
build(format = c("html", "pdf"))
```

Si no indica `format`, `build()` utiliza los perfiles declarados por el proyecto y construye aquellos para los que existe renderer.

En un proyecto `type: quarto` sin `publication.strategy`, IASI delega el renderizado en Quarto. Si se indica explícitamente `format` en ese modo, se admite como máximo un formato por llamada.

`build()` no utiliza el estado anterior para decidir si debe trabajar: una llamada a `build()` significa construir.

6 Construir y publicar

6.1 Operaciones principales

La API pública del paquete `iasi` gira alrededor de cuatro operaciones:

```
validate()  
build()  
publish()  
release()
```

Todas parten del mismo mecanismo de descubrimiento. A partir de `path`, IASI recorre el árbol, localiza proyectos marcados por `_iasi.yml` o `.iasi.yml` y resuelve su configuración efectiva. Los proyectos `repository` actúan como contenedores y no se procesan directamente.

6.1.1 Validar

```
validate()
```

`validate()` comprueba los proyectos IASI seleccionados.

Para proyectos Quarto valida su configuración y, cuando existe una estrategia IASI, también la estructura editorial correspondiente. Para paquetes R comprueba, entre otras condiciones, la existencia de `DESCRIPTION`.

No construye, publica ni libera artefactos.

6.1.2 Construir

```
build()
```

`build()` construye siempre que se invoca. No intenta decidir si los fuentes han cambiado.

Para proyectos Quarto, los resultados se materializan bajo `_outputs/`. Cuando se utiliza el pipeline IASI, los perfiles se mantienen separados:

```
publication/  
  _outputs/  
    html/  
    pdf/  
    epub/  
    ...
```

Durante la edición puede construirse un formato concreto:

```
build(format = "html")
```

Para un proyecto `r-package`, `build()` genera los artefactos del paquete en su raíz:

```
package/  
  package_version.tar.gz  
  package_version.zip
```

El primero es el paquete fuente y el segundo el paquete binario en Windows.

6.1.3 Publicar

```
publish()
```

`publish()` trabaja con proyectos IASI `website` y con proyectos `quarto` que hayan declarado `publication.strategy`.

Toma como entrada exclusivamente formatos ya contruidos en `_outputs/`; **no vuelve a renderizar las fuentes**.

Puede publicar todos los formatos contruidos:

```
publish()
```

o seleccionar uno:

```
publish(format = "html")
```

IASI solo considera formatos los perfiles declarados por el proyecto Quarto. Directorios residuales o arbitrarios dentro de `_outputs/` no se promocionan a formatos publicables.

La preparación se realiza en un directorio temporal único situado junto al destino final. Solo cuando la copia, el postprocesado y los metadatos terminan correctamente se sustituye `_publish/`. Si el proceso falla antes, la publicación anterior permanece intacta.

En una publicación individual:

```
publication/  
  _outputs/  
  _publish/  
    index.html  
    chapters/  
    pdf/  
      libro.pdf  
    epub/  
      libro.epub
```

El contenido HTML ocupa la raíz del árbol publicable cuando es el perfil primario; los demás formatos permanecen como exports.

La metadata `.publish` conserva el timestamp y una firma del contenido utilizado para materializar la publicación.

6.1.4 Liberar

```
release()
```

`release()` reúne entregables locales. No despliega nada en servicios externos.

Para un `r-package`, mueve a `release/` los artefactos contruidos (`*.tar.*` y `*.zip`).

Para un `website` o un proyecto `quarto` con estrategia, reúne el contenido previamente materializado por `publish()`.

Para un proyecto `quarto` sin estrategia, `publish()` no aplica y `release()` recoge directamente los artefactos de `_outputs/`.

Por tanto, el ciclo depende del tipo de proyecto:

```
Quarto con estrategia / Website  
validate → build → publish → release
```

```
Quarto as-is  
validate → build → release
```

```
R package  
validate → build → release
```

IASI R termina aquí. El despliegue a sistemas externos pertenece a herramientas de nivel superior, como `iasi-dev`.

6.2 Flujo recomendado

Durante la edición de un libro con estrategia IASI, construya únicamente el formato que necesita revisar:

```
build(format = "html")
```

Cuando quiera materializar todos los perfiles declarados:

```
build()
```

Cuando los resultados construidos deban convertirse en una publicación local:

```
publish()
```

Si solo quiere publicar un formato ya construido:

```
publish(format = "html")
```

Cuando quiera reunir los entregables locales:

```
release()
```

La regla práctica es:

```
build    → construye  
publish  → materializa _publish a partir de _outputs  
release  → reúne entregables locales
```

`publish()` no es incremental: si se invoca sobre formatos disponibles, vuelve a preparar la publicación. La firma guardada en `.publish` describe la materialización realizada, pero no evita por sí sola una nueva ejecución.

7 Multiproyectos

7.1 Trabajar con varios proyectos IASI

Un repositorio puede contener varios proyectos IASI bajo una misma raíz. La raíz puede declararse como contenedor mediante:

```
type: repository
```

Los directorios intermedios no necesitan configuración IASI: el descubrimiento continúa descendiendo hasta encontrar un directorio que contenga `_iasi.yml` o `.iasi.yml`, salvo las ramas excluidas por el motor.

Ejemplo:

```
repository/  
  _iasi.yml          type: repository  
  docs/  
    user-guide/  
      _iasi.yml      type: quarto  
      _quarto.yml    project.type: book  
      chapters/  
  software/  
    iasi-r/  
      _iasi.yml      type: r-package  
      DESCRIPTION
```

En este ejemplo, `docs/` y `software/` son únicamente contenedores. Los proyectos IASI están en la raíz, en `docs/user-guide/` y en `software/iasi-r/`.

7.2 Configuración heredada

La configuración local de cada proyecto tiene prioridad. La configuración de sus padres completa únicamente los valores que falten.

Por ejemplo, la raíz puede fijar un destino común de release:

```
type: repository

paths:
  release: release
```

Los proyectos hijos pueden heredar ese destino sin repetirlo.

7.3 path define el ámbito

Las operaciones utilizan `path` como selector del ámbito:

```
validate(path = ".")
build(path = ".")
publish(path = ".")
release(path = ".")
```

Si se ejecuta una operación desde una subcarpeta, el descubrimiento se limita a ese árbol.

No existe un selector adicional `book:` para trabajar sobre un proyecto concreto, utilice su `path`.

Por ejemplo:

```
build(path = "docs/user-guide", format = "html")
```

7.4 Aplicabilidad de las operaciones

Descubrir un proyecto no significa que todas las operaciones le apliquen del mismo modo.

<code>repository</code>	→ contenedor; no se procesa directamente
<code>quarto + strategy</code>	→ <code>validate</code> , <code>build</code> , <code>publish</code> , <code>release</code>
<code>quarto sin strategy</code>	→ <code>validate</code> , <code>build</code> , <code>release</code>
<code>website</code>	→ <code>validate</code> , <code>build</code> , <code>publish</code> , <code>release</code>
<code>r-package</code>	→ <code>validate</code> , <code>build</code> , <code>release</code>

7.5 Publicación local por proyecto

Cada proyecto publicable posee su propio `_publish/`:

```
repository/  
  docs/  
    01-user-guide/  
      _outputs/  
      _publish/  
    02-developer-guide/  
      _outputs/  
      _publish/  
  release/
```

`publish()` nunca necesita un `_publish/` compartido para el multiproyecto. Cada proyecto prepara y sustituye únicamente su propia publicación local.

7.6 `release()` reúne los entregables

El ensamblado común ocurre en `release()`.

Para proyectos Quarto, el nombre del destino elimina un prefijo numérico organizativo:

```
01-user-guide/_publish/      → release/user-guide/  
02-developer-guide/_publish/ → release/developer-guide/
```

De esta manera se separan claramente los niveles:

```
fuentes  
  ↓  
_outputs/      artefactos contruidos de cada proyecto  
  ↓  
_publish/      publicación local de cada proyecto  
  ↓  
release/       entregables reunidos en el ámbito superior
```

8 Problemas frecuentes

8.1 El proyecto no se reconoce

Compruebe que se encuentra en la ruta correcta y que el proyecto contiene `_iasi.yml` o `.iasi.yml`.

El campo `type` debe estar en la raíz de la configuración y utilizar uno de los tipos soportados:

```
repository
quarto
website
r-package
```

Para un proyecto Quarto también debe existir `_quarto.yml`; para un `r-package`, `DESCRIPTION`.

8.2 Hay dos archivos de configuración IASI

Un mismo directorio no debe contener simultáneamente `_iasi.yml` y `.iasi.yml`. Si existen ambos, la configuración es ambigua.

Conserve únicamente el que corresponda al proyecto.

8.3 Un proyecto hijo no se descubre

Los directorios intermedios no necesitan una marca IASI. El descubrimiento desciende por el árbol hasta encontrar proyectos, excepto dentro de ramas excluidas.

Compruebe que el directorio del proyecto contiene directamente `_iasi.yml` o `.iasi.yml` y que no está bajo una ruta excluida.

8.4 Aparece “No IASI publication strategy defined”

En un proyecto:

```
type: quarto
```

sin `publication.strategy`, ese aviso significa que IASI utilizará **Quarto as-is**. Es un modo válido.

Si esperaba utilizar una estrategia IASI, declárela dentro de `publication`:

```
type: quarto
publication:
  strategy: parted
```

No coloque `strategy` en la raíz. Esa ubicación es obsoleta y se considera un error de configuración.

8.5 Un formato no se genera

Compruebe que el perfil está declarado por el proyecto y que IASI dispone de renderer para él.

Por ejemplo:

```
_quarto-pdf.yml
```

También debe disponer de las dependencias que necesite el formato. PDF, por ejemplo, puede requerir una instalación TeX funcional.

8.6 Quiero reconstruir

Utilice:

```
build()
```

`build()` construye siempre que se invoca.

8.7 `publish()` dice que no hay formatos contruidos

`publish()` solo utiliza perfiles declarados por Quarto que tengan una materialización disponible en `_outputs/`.

Si ha cambiado la configuración o existen restos de un build anterior, puede borrar `_outputs/` y volver a construirlo:

```
build()
publish()
```

Los directorios residuales que no correspondan a perfiles declarados se ignoran como formatos de publicación.

8.8 Quiero publicar lo que ya está construido

Para proyectos publicables utilice:

```
publish()
```

`publish()` toma su entrada de `_outputs/` y no vuelve a renderizar.

En un `type: quarto` sin estrategia, `publish()` no aplica; utilice `release()` sobre los outputs contruidos.

8.9 Quiero preparar los entregables locales

Utilice:

```
release()
```

Para paquetes R recoge los artefactos fuente y binario. Para publicaciones recoge la materialización correspondiente según el tipo de proyecto.

8.10 Quiero desplegar a un servicio externo

El despliegue externo no forma parte de IASI R. Utilice la herramienta responsable del entorno de desarrollo o despliegue, por ejemplo `iasi-dev`.